Лабораторна робота №9

Тема: React State та Effects: Context API, useReducer, useEffect і Custom Hooks.

Мета: навчитися організовувати стан React-застосунку, коли одного локального `useState` стає недостатньо. Опанувати підняття стану, Context API, `useReducer`, побічні ефекти, роботу з `localStorage` та створення власних hooks.

Лабораторна є продовженням лабораторної роботи №8. Необхідно продовжити розробку застосунку **Study Planner** без втрати попереднього функціоналу.

Рекомендована feature-гілка: `feature/state-effects`.

Вимоги до звіту

1. Використовувати React 19.x та Node.js 22.22 або новішу версію.
2. Використовувати TypeScript.
3. Використовувати функціональні компоненти та hooks.
4. Використати Context API для спільного стану.
5. Використати `useReducer` щонайменше для одного набору пов'язаних змін стану.
6. Використати `useEffect` для роботи з побічним ефектом.
7. Реалізувати збереження даних у `localStorage`.
8. Винести повторно використовувану логіку в Custom Hook.
9. Для стилізації використати один із підходів: SCSS із дотриманням BEM, Bootstrap або Tailwind CSS.
10. Якщо обрано SCSS, дотримуватися BEM-патерну для класів.
11. Дотримуватися ESLint та єдиного стилю коду.
12. Код повинен бути розміщений в особистому GitHub репозиторії студента.
13. Дотримуватися Conventional Commits.

Теоретична частина

Опрацювати:

1. Managing State.
2. Sharing State Between Components.
3. Extracting State Logic into a Reducer.
4. Scaling Up with Reducer and Context.
5. Synchronizing with Effects.
6. Reusing Logic with Custom Hooks.

Стилізація та BEM

Для стилізації обрати один із трьох підходів:

1. **SCSS + BEM** — власні стилі з класами за схемою Block__Element—Modifier;
2. **Bootstrap** — готові utility та component класи;
3. **Tailwind CSS** — utility-first класи.

Не потрібно поєднувати всі три підходи одночасно.

Якщо обрано **SCSS + BEM**, стилі повинні бути організовані послідовно, а класи — відповідати BEM.

Практична частина

1. Розвиток Study Planner

Продовжити проєкт з лабораторної роботи №8. Не видаляти вже реалізований функціонал.

У застосунку повинні залишитися:

- створення завдання;
- зміна стану виконання;
- видалення;
- фільтрація;
- статистика.

2. Підняття стану

Проаналізувати компоненти `TaskForm`, `TaskList`, `TaskFilters` та `TaskStats`.

State, який використовується кількома компонентами, винести на спільний рівень. Заборонено дублювати один і той самий стан у кількох незалежних компонентах.

Продемонструвати в коді передачу даних і callback-функцій через props там, де це доречно.

3. useReducer

Перенести основні операції над завданнями у reducer.

Створити тип дій щонайменше для:

- `add`;
- `toggle`;

- `remove`.

За потреби додати:

- `update`;
- `clearCompleted`;
- `reset`.

Reducer не повинен містити UI-логіку або звернення до DOM.

4. Відповідальність модулів

Під час рефакторингу розподіліть відповідальність між модулями:

- reducer відповідає за переходи стану;
- Context відповідає за спільний доступ до стану та дій;
- компоненти відповідають за відображення та взаємодію з користувачем;
- Custom Hook інкапсулює повторно використовувану логіку.

Не потрібно переносити будь-яку логіку в окремий файл лише заради кількості файлів.

5. Context API

Створити `TaskContext` та `TaskProvider`.

Через Context зробити доступними:

- список завдань;
- поточний фільтр;
- дії зміни стану.

Не потрібно передавати всі ці значення через декілька рівнів props.

Компоненти, які не використовують конкретне значення Context, не повинні отримувати його через props лише «про запас».

6. localStorage та useEffect

Реалізувати збереження списку завдань у `localStorage`.

У цій задачі `localStorage` є зовнішнім сховищем, синхронізацію з яким потрібно виконувати через Effect.

Рекомендований сценарій:

1. під час ініціалізації state прочитати збережені дані, якщо вони є;

2. якщо збережених даних немає або вони пошкоджені, використати початковий список;
3. після зміни списку завдань записати актуальне значення в `localStorage` за допомогою `useEffect`;
4. не використовувати `useEffect` для звичайних обчислень, які можна виконати під час render.

7. Custom Hook

Створити власний hook для роботи з `localStorage`, наприклад:

```
useLocalStorage<T>(key, initialValue)
```

Hook повинен уміти:

- читати початкове значення;
- повертати актуальне значення;
- оновлювати значення;
- серіалізувати та десеріалізувати дані;
- безпечно обробляти відсутність або пошкодження збереженого значення.

Не потрібно приховувати в Custom Hook усю логіку застосунку: його відповідальність — повторно використовувана робота з `localStorage`.

За бажанням цей hook можна зробити універсальним і використати не лише для tasks.

8. Фільтр та derived state

Поточний список завдань для відображення не потрібно дублювати в окремому state.

Фільтрований список повинен обчислюватися з основного state.

Статистика також повинна обчислюватися з актуального списку завдань, а не з окремих лічильників, які легко розсинхронізувати.

9. UX-стани

Додати до інтерфейсу:

- повідомлення, коли завдань немає;
- індикацію активного фільтра.

У цій лабораторній не потрібно створювати окрему систему сповіщень.

10. Додаткові завдання

За бажанням також можна додати:

1. повідомлення про успішне додавання, оновлення або видалення;
2. простий `Toast` або inline-компонент для таких повідомлень.
3. Перемикач «Показувати тільки високий пріоритет».
4. Масове завершення усіх невиконаних завдань.
5. Відновлення останньої видаленої задачі.
6. Збереження активного фільтра у `localStorage`.
7. Окремий `ThemeContext` для світлої/темної теми.

Рекомендована структура

```
src/
 ├──── components/
 ├──── context/
 │     └──── TaskContext.tsx
 ├──── hooks/
 │     └──── useLocalStorage.ts
 ├──── reducers/
 │     └──── taskReducer.ts
 ├──── types/
 ├──── data/
 ├──── App.tsx
 └──── main.tsx
```

Структура може бути змінена, якщо відповідальність модулів залишається зрозумілою. Не потрібно створювати окремий модуль для кожної функції або будувати складну feature-first/FSD-архітектуру: у цій лабораторній головною метою є організація state та side effects.

Демонстрація на захисті

За час захисту необхідно продемонструвати:

1. зміну стану завдання через reducer;
2. роботу одного компонента, який отримує дані через Context;
3. зміну завдання та перезавантаження сторінки, після якого дані залишаються;
4. у коді показати `useReducer`, `useEffect` та Custom Hook.

Не потрібно демонструвати всі додаткові можливості.

Приклад Conventional Commit:

```
feat(lab-09): add reducer context and persistence
```

Контрольні запитання

1. Коли state доцільно піднімати у батьківський компонент?
2. Що таке prop drilling і яку проблему вирішує Context API?
3. Що таке reducer? Чим reducer відрізняється від звичайної функції оновлення state?
4. Якою повинна бути чиста reducer-функція?
5. Що таке action і dispatch?
6. Коли `useReducer` доцільніше використовувати за `useState`?
7. Що таке Context API і з яких основних частин складається його використання?
8. Чи потрібно через Context передавати весь state застосунку?
9. Що таке side effect у React?
10. Для чого використовується `useEffect`?
11. Коли запускається effect і коли виконується cleanup?
12. Чому запис у `localStorage` є побічним ефектом?
13. Чому не варто дублювати derived state?
14. Що таке Custom Hook?
15. Які правила потрібно дотримуватися при створенні власних hooks?